

Effects of Sodium Isotopes on the Neutronic Properties of Sodium-Cooled Fast Reactors (SFR)

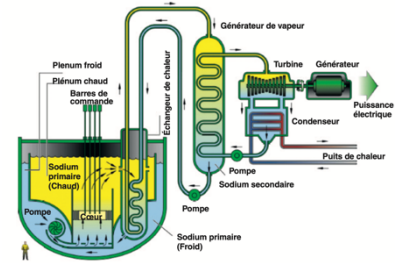
D.QUILLAUD – February 13, 2025

I. Introduction

Sodium-cooled fast reactors (SFR) play a crucial role in closing the fuel cycle and managing minor actinides. Sodium (Na) is used as a coolant primarily due to its excellent thermophysical properties. However, other constraints of a neutronic nature must also be taken into consideration: the primary coolant circulating through the reactor core is exposed to the neutron flux. The coolant must neither absorb these neutrons nor slow them down. Over the long term, additional considerations arise:

- SFRs must operate over long periods without frequent refueling (unlike PWRs).
- The reactivity stability must be ensured to guarantee safe and optimal operation over several decades.

With this in mind, the question of the influence of sodium isotopes, both at beginning-of-life and after activation, on the neutronic properties of the reactor remains essential for the design and safety of SFRs.



II. Problem Statement

1. Context and Formulation

At beginning-of-life, natural sodium ^{23}Na , a stable isotope, is introduced. It will produce activated sodium ^{24}Na , with a half-life of 15 hours. The presence of other unstable and/or trace isotopes (^{22}Na , ^{21}Na , ^{20}Na) is also noted.

Both isotopes have a low capture cross-section in the fast energy range:

Isotope	Microscopic capture cross-section (barn) 1 MeV – 293K - ENDF IAEA n, γ calculator
^{23}Na	1.65091E-4
^{24}Na	2.3E-4

These could have an impact on the long-term reactivity of the reactor. The central question of this study is the following:

To what extent does neutron absorption by sodium influence the neutronics of the reactor ?

III. Experimentation and Modeling

a. Resolution Approach

In order to best estimate the impact of sodium on core neutronics, three parameters are proposed for study. To quantify their variation, these parameters will be examined as a function of the sodium void coefficient within the assembly environment (inter-pin and outer zones). This is expressed as a fraction of inert gas mixed into the sodium composition, in this case Helium-4, chosen for its stability and near-zero neutron absorption capacity.

A void coefficient sweep allows us both to quantify the actual impact of sodium on the reactor by comparing void fractions against the nominal 100% sodium case, and to quantify the "void effect", that is the reactor's ability to be self-stabilizing in the event of partial or total coolant loss (particularly during thermal transients or voiding accidents)

1. Study of the Effect of Sodium on k-eff:

The k-eff study provides a global picture of the sodium effect. The temperature sweep will show how sodium neutron absorption varies with temperature and whether an increase in temperature amplifies or attenuates the reactivity loss.

A burnup study is also planned to evaluate how the reactivity loss interacts with fuel depletion, and whether the impact of sodium neutron absorption is constant or evolves over the cycle.

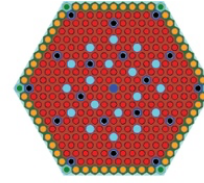
2. Study of the energy dependence of neutron capture:

The objective of this analysis is to characterize how sodium influences the spectral distribution of neutrons and to quantify the fraction of neutrons absorbed by sodium. This analysis will also provide precise data on the critical energies at which sodium absorption is significant.

b. Assumptions and Model Description

The Monte Carlo code OpenMC is used here to perform various analyses and comparisons, drawing inspiration from the Superphénix SFR architecture.

A complete MOX assembly is modeled, providing more faithful neutronic representativity by incorporating self-shielding, leakage, and fuel-coolant interaction effects.



The geometry is a hexagonal prism with an assembly pitch of 23.6 cm and a cell pitch of 1.275 cm. Two distinct axial planes with reflective boundary conditions, equivalent to an infinite geometry along the z-axis, have been defined. All dimensions used are available in the script in the Appendix.

- Central tube cell
- Guide tube cell
- Fuel Cell with the enrichment 3.6% wt. on ^{235}U and 4% wt. on Gd_2O_3
- Fuel Cell with 3.6% wt. enrichment of fissile plutonium
- Fuel Cell with 2.7% wt. enrichment of fissile plutonium
- Fuel Cell with 2.4% wt. enrichment of fissile plutonium

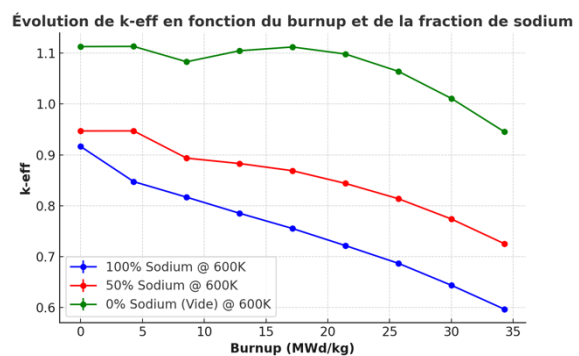
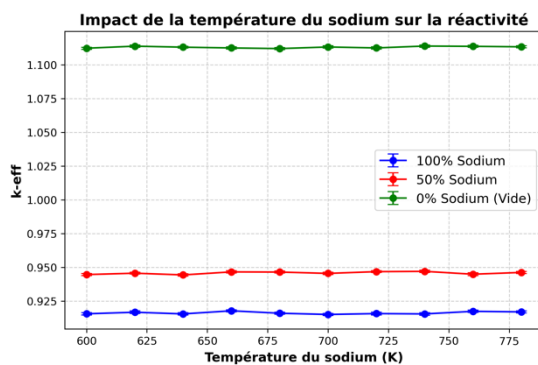
As nominal conditions, a temperature of 600 K for the sodium, 900K for the cladding and helium gap and 1200 K for the fuel are chosen. Dimensions:

All calculations are performed in critical mode (k-eff search) with convergence over 100 iterations and source sampling of 10,000 particles.

c. Study of the Effect of Sodium on k-eff

A temperature range of 600K to 800K is assumed to cover the various operating conditions. The burnup evolution spans a range up to 35 MWd/kg, corresponding to approximately 2 months of constant nominal power operation on a Superphénix-type SFR (300 MWth).

It is notable that the void coefficient is the dominant effect on these



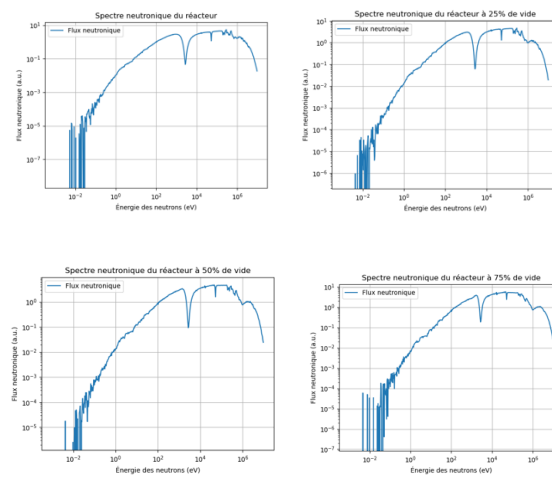
evolutions. A reactivity gain of approximately 3% is nonetheless observed at 50% voiding, over a temperature sweep, which appears to be of limited significance in the context of such a voiding accident.

As expected, k_{eff} decreases regardless of the situation with increasing burnup. However, sodium tends to "smooth" the reactivity of the system as the fuel ages, but the reactivity loss due to burnup is faster with sodium.

Temperature effects on neutronics appear marginal and are difficult to analyze beyond a linear trend.

In light of this first study, full voiding will be excluded from the subsequent analysis.

d. Study of the Energy Dependence of Neutron Capture



On the general shape of the spectrum:

- At high energy ($> 10^5$ eV): Little visible effect on neutron flux intensity with sodium voiding
- At low energy (< 10 eV): A decrease in neutron flux is observable with increasing void fraction.
- In the intermediate region (10 eV - 10^5 eV): Modifications are observed in the dips associated with absorption resonances.

The absorption resonance dips of plutonium and uranium-238 become more pronounced with decreasing void fraction. This is explained by the fact that more neutrons are absorbed by sodium, leaving fewer neutrons to interact within the actinide resonances.

The absence of sodium does not drastically alter the spectrum at high energies, but noticeably reduces thermalization and observably modifies absorption in the resonance region.

The presence of sodium maintains a low level of thermalization, but near-total voiding makes the spectrum even more "fast".

IV. Discussion and Conclusion

Neutron absorption by sodium influences reactor neutronics primarily through its effect on the void coefficient and the modification of the neutron spectrum. Partial sodium voiding results in a reactivity gain of approximately 3% at 50% void, although this effect appears marginal in the context of a voiding accident. The k -eff evolution follows the expected decreasing trend with burnup, but the presence of sodium tends to smooth the reactivity loss while slightly accelerating it.

The spectral study shows that the progressive removal of sodium enhances the absorption resonances of plutonium and uranium-238, due to the reduction in parasitic absorption. Sodium plays a minor role in thermalization, but its removal further hardens (more "fast") the spectrum further, which impacts reactivity. However, temperature effects remain marginal and difficult to quantify beyond a linear trend.

These results are subject to certain uncertainties, notably the temperature interpolation of cross-sections by OpenMC (the ENDF/B used provides cross-sections at 294K, 600K, 900K and 1200K), the analysis on an isolated assembly, and the limited representativity of a standalone MOX assembly. To refine the study, possible improvements could include a comparison with an alternative coolant such as lead, a full-core model, and a study extended to various operating points.

V. References

<https://www.tandfonline.com/doi/full/10.1080/00295639.2019.1614367>
https://www.cea.fr/Documents/monographies/combustibles-nucléaires_réacteurs-eau.pdf
<https://www.cea.fr/Documents/monographies/reacteurs-nucleaires-sodium-pourquoi-reacteurs-rapides-sodium.pdf>
<https://iopscience.iop.org/article/10.1088/1742-6596/1689/1/012056/pdf>
<https://nucet.pensoft.net/article/91090/>

VI. Appendix

```

import openmc
import openmc.deplete
import numpy as np
import matplotlib.pyplot as plt
from math import sqrt

# =====
# Material definitions
# =====

# MOX fuel (UGD-PuO2)
UGD_fuel = openmc.Material(name='UGD Fuel')
UGD_fuel.add_nuclide('U238', 0.924)
UGD_fuel.add_nuclide('U235', 0.036)
UGD_fuel.add_element('O', 1.04)
UGD_fuel.set_density('g/cm3', 10.5)
UGD_fuel.temperature = 1200 # Temperature in K
UGD_fuel.volume = 144.96

def mox_fuel(enrichment):
    """Create a MOX material with a given Pu239 enrichment."""
    mox = openmc.Material()
    mox.add_element('U238', 0.91)
    mox.add_nuclide('Pu239', enrichment)
    mox.add_nuclide('Pu240', 0.035)
    mox.add_nuclide('Pu241', 0.005)
    mox.add_nuclide('Pu242', 0.005)
    mox.add_nuclide('O16', 2.00)
    mox.set_density('g/cm3', 10.5)
    mox.temperature = 1200
    mox.volume = 144.96
    return mox

mox_3_6 = mox_fuel(0.036)
mox_2_7 = mox_fuel(0.027)
mox_2_4 = mox_fuel(0.024)

# Sodium coolant
sodium = openmc.Material(name='Sodium')
sodium.add_nuclide('Na23', 1.0)
sodium.set_density('g/cm3', 0.4635)
sodium.temperature = 600 # Temperature in K

# Zirconium cladding
zirconium_cladding = openmc.Material(name='Zirconium Cladding')
zirconium_cladding.add_element('Zr', 1.0)
zirconium_cladding.set_density('g/cm3', 6.5)
zirconium_cladding.temperature = 700 # Temperature in K

# Helium gas gap
gas_gap = openmc.Material(name='Helium Gap')
gas_gap.add_element('He', 1.0)
gas_gap.set_density('g/cm3', 0.0001786)
gas_gap.temperature = 900 # Temperature in K

# Material collection
materials = openmc.Materials([UGD_fuel, mox_3_6, mox_2_7, mox_2_4, sodium, gas_gap, zirconium_cladding])
materials.export_to_xml()

# =====
# Geometry definition
# =====

pin_pitch = 1.25984
fuel_radius = openmc.ZCylinder(r=0.39218)
clad_inner_radius = openmc.ZCylinder(r=0.40005)
clad_outer_radius = openmc.ZCylinder(r=0.45729)
#outzone_sodium = openmc.ZCylinder(r=pin_pitch/2)

N = 11 # Number of pin rings
assembly_width = (N+1)*pin_pitch*sqrt(3)
assembly_pitch = assembly_width + 0.5

pin_box = openmc.model.HexagonalPrism(edge_length=pin_pitch*2, boundary_type='reflective')
assembly_box = openmc.model.HexagonalPrism(edge_length=30, boundary_type='reflective')

mox_3_6_cell = openmc.Cell(name='Fuel1', fill=mox_3_6, region=-fuel_radius)
mox_2_7_cell = openmc.Cell(name='Fuel2', fill=mox_2_7, region=-fuel_radius)
mox_2_4_cell = openmc.Cell(name='Fuel3', fill=mox_2_4, region=-fuel_radius)

UGD_cell = openmc.Cell(name='UGd', fill=UGD_fuel, region=-fuel_radius)
guide_tube_cell = openmc.Cell(name='Guide Tube', fill=zirconium_cladding, region=-clad_outer_radius)

gap_3_6_cell = openmc.Cell(name='Helium 3.6 Gap', fill=gas_gap, region=+fuel_radius & -clad_inner_radius)
gap_2_7_cell = openmc.Cell(name='Helium 2.7 Gap', fill=gas_gap, region=+fuel_radius & -clad_inner_radius)
gap_2_4_cell = openmc.Cell(name='Helium 2.4 Gap', fill=gas_gap, region=+fuel_radius & -clad_inner_radius)
gap_Ugd_cell = openmc.Cell(name='Helium Ugd Gap', fill=gas_gap, region=+fuel_radius & -clad_inner_radius)

clad_3_6_cell = openmc.Cell(name='Cladding 3.6', fill=zirconium_cladding, region=+clad_inner_radius & -clad_outer_radius)
clad_2_7_cell = openmc.Cell(name='Cladding 2.7', fill=zirconium_cladding, region=+clad_inner_radius & -clad_outer_radius)
clad_2_4_cell = openmc.Cell(name='Cladding 2.4', fill=zirconium_cladding, region=+clad_inner_radius & -clad_outer_radius)
clad_Ugd_cell = openmc.Cell(name='Cladding Ugd', fill=zirconium_cladding, region=+clad_inner_radius & -clad_outer_radius)

sodium_inner_3_6_cell = openmc.Cell(name='Coolant 3.6 inner', fill=sodium, region=+clad_outer_radius)
sodium_inner_2_7_cell = openmc.Cell(name='Coolant 2.7 inner', fill=sodium, region=+clad_outer_radius)
sodium_inner_2_4_cell = openmc.Cell(name='Coolant 2.4 inner', fill=sodium, region=+clad_outer_radius)
sodium_inner_Ugd_cell = openmc.Cell(name='Coolant Ugd inner', fill=sodium, region=+clad_outer_radius)
sodium_inner_GT_cell = openmc.Cell(name='Coolant GT inner', fill=sodium, region=+clad_outer_radius)

sodium_outer_cell = openmc.Cell(name='Coolant outer', fill=sodium)

sodium_universe = openmc.Universe(cells=[sodium_outer_cell])
guide_tube_universe = openmc.Universe(cells=[guide_tube_cell, sodium_inner_GT_cell])

mox_3_6_universe = openmc.Universe(cells=[mox_3_6_cell, gap_3_6_cell, clad_3_6_cell, sodium_inner_3_6_cell])
mox_2_7_universe = openmc.Universe(cells=[mox_2_7_cell, gap_2_7_cell, clad_2_7_cell, sodium_inner_2_7_cell])
mox_2_4_universe = openmc.Universe(cells=[mox_2_4_cell, gap_2_4_cell, clad_2_4_cell, sodium_inner_2_4_cell])
UGD_universe = openmc.Universe(cells=[UGD_cell, gap_Ugd_cell, clad_Ugd_cell, sodium_inner_Ugd_cell])

lattice = openmc.HexLattice(lattice_id=200)
lattice.center = (0.0, 0.0)
lattice.pitch = [pin_pitch]
lattice.outer = sodium_universe
lattice.universes = [
    (mox_2_4_universe) + (mox_2_7_universe) * 9 * 6,
    (UGD_universe) + (mox_3_6_universe) * 8 * 6,
    (mox_3_6_universe) * 4 + (UGD_universe) + (mox_3_6_universe) * 3 * 6,
    (mox_3_6_universe) * 7 * 6,
    (mox_3_6_universe) * 3 + (guide_tube_universe) + (mox_3_6_universe) * 2 * 6,
    (guide_tube_universe) + (mox_3_6_universe) * 4 * 6,
    (mox_3_6_universe) * 3 + (UGD_universe) * 6,
    (guide_tube_universe) + (mox_3_6_universe) + (mox_3_6_universe) * 6,
    (mox_3_6_universe) * 6,
    (guide_tube_universe)
]

assembly_cell = openmc.Cell(fill=lattice, region=-assembly_box)
root_universe = openmc.Universe(cells=[assembly_cell])
geometry = openmc.Geometry(root_universe)
geometry.export_to_xml()

# =====
# Physics settings
# =====

source = openmc.Source()
source.space = openmc.stats.Point(
    (0, pin_pitch, 0.0) # Source point
)
source.angle = openmc.stats.Isotropic()
source.energy = openmc.stats.Wien(a=0.988, b=2.249)

settings = openmc.Settings()
settings.batches = 50
settings.inactive = 10
settings.particles = 10000
settings.source = source
settings.run_mode = 'eigenvalue'
settings.temperature = ('method': 'interpolation', 'range': (300, 1200)) # Cross-section interpolation for temperatures not available in ENDF/B-VII.1
settings.export_to_xml()

```

